

Device driver nomenclature

0 Unnamed devices (e.g., non-device mounts)

0 = reserved as null device number

See block major 144, 145, 146 for expansion areas.

1 char Memory devices

1 = /dev/mem.....Physical memory access

2 = /dev/kmem.....Kernel virtual memory access

3 = /dev/null.....Null device

4 = /dev/port.....I/O port access

5 = /dev/zero.....Null byte source

6 = /dev/core.....OBSOLETE - replaced by /proc/kcore

7 = /dev/full.....Returns ENOSPC on write

8 = /dev/random.....Nondeterministic random number gen.

9 = /dev/urandom.....Faster, less secure random number gen.

10 = /dev/aio.....Asynchronous I/O notification interface

11 = /dev/kmsg.....Writes to this come out as printk's

1 block RAM disk

0 = /dev/ram0.....First RAM disk

1 = /dev/ram1.....Second RAM disk

...

250 = /dev/initrd.....Initial RAM disk (2.6)

Older kernels had /dev/ramdisk (1, 1) here /dev/initrd refers to a RAM disk which was preloaded by the boot loader; newer kernels use /dev/ram0 for the initrd.

2 char Pseudo-TTY masters

0 = /dev/ptyp0.....First PTY master

1 = /dev/ptyp1.....Second PTY master

...

255 = /dev/ptyef.....256th PTY master

2 block Floppy disks

0 = /dev/fd0.....Controller 0, drive 0, autodetect

1 = /dev/fd1.....Controller 0, drive 1, autodetect

2 = /dev/fd2.....Controller 0, drive 2, autodetect

3 = /dev/fd3.....Controller 0, drive 3, autodetect

128 = /dev/fd4.....Controller 1, drive 0, autodetect

129 = /dev/fd5.....Controller 1, drive 1, autodetect

130 = /dev/fd6.....Controller 1, drive 2, autodetect

131 = /dev/fd7.....Controller 1, drive 3, autodetect

and so on ...

Continued on next page...

```
/* This function is required to activate all four slots on
the UCR-6152B
```

```
* flash reader */
int usb_xstor_ucr6152b_init(struct usb_data *us)
{
    struct bulk_cb_wrap *bcb = (struct bulk_cb_wrap*) us->tobuf;
    struct bulk_cs_wrap *bcs = (struct bulk_cs_wrap*) us->toobuf;
    int res, partial;
    static char init_string[] = "\x00\x00\x00";
    x00EPCCHPS";

```

```
US_DEBUGP("Sending UCR-6152B initialization
packet.\n");

```

```
bcb->Signature = cpu_to_le32(US_BULK_CB_SIGNATURE);
bcb->Tag = 0;
bcb->DataTransferLength = cpu_to_le32(0);
bcb->Flags = bcb->Len = 0;
bcb->Length = sizeof(init_string) - 1;
memset(bcb->CDR, 0, sizeof(bcb->CDR));
memcpy(bcb->CDR, init_string, sizeof(init_string)
- 1);

```

```
res = usb_xstor_bulk_transfer(bufus, us->send,
bulk_pipe, bcb,

```

```
US_BULK_CB_WRAP_LEN, &partial);
if(res)
    return res;

```

```
US_DEBUGP("Getting status packet.\n");
res = usb_xstor_bulk_transfer(bufus, us->recv,
bulk_pipe, bcs,

```

```
US_BULK_CS_WRAP_LEN, &partial);

```

```
return (res ? -1 : 0);
}

```

We can meddle with all these when we begin our experimental session!

In case the DD is already registered under a particular major number and the corresponding file operation does not match with those, then the `register_chrdev()` function will return a negative value.

You may come across two types of devices—viz., block-oriented and character-oriented devices. In the case of the first set of devices, any given block can be read or written to at the will of the programmer (i.e., they support random access). This task is done using cache. This feature